



Отчет по лабораторной работе

Задание 1 по курсу «Информационные технологии» вариант 11

Учебная группа

М7О-207БВ-24

Студент

Никитин П.А. _____

Преподаватель

Барчев Н.Б.

Содержание

1 Текст задания лабораторной работы	1
2 Краткое описание методов решения поставленных задач	2
3 Псевдокод	3
4 Сведения о программной реализации	5
4.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования	5
4.2 Описание входных и выходных данных	7
4.3 Краткое описание спроектированных программных единиц	7
4.4 Список диагностических сообщений с указанием возможных причин их появления	10
5 Инструкция использования программы	12
6 Листинг программной разработки	15
7 Результаты тестирования	23
8 Дальнейшее развитие программы.	27

1

Текст задания лабораторной работы

Задание 1 по курсу «Информационные технологии» вариант 11

1. Подготовить программу, формирующую на основе информации, вводимой пользователем с клавиатуры, два внешних текстовых файла:
 - файл автомобилей: состоит из записей, каждая из которых включает три поля – марки, модели и номерного знака;
 - файл регистрационных сведений: состоит из записей, каждая из которых включает четыре поля – номерного знака, фамилии и адреса владельца, года выпуска.
2. В процессе проектирования предусмотреть необходимые по смыслу задания проверки корректности данных, а также адекватное задаче взаимодействие с пользователем. **Учесть, что ограничения на размеры файлов отсутствуют, файлы в общем случае могут иметь различные длину.**
В обязательном порядке использовать языковые средства организации программных единиц.
Не использовать программные средства, поддерживающие работу с базами данных.
3. Программу подготовить с использованием средств одной из реализаций языка программирования C++, **ввод и вывод информации реализовать при помощи средств языка программирования C.** Выполнить тестирование обеих программ согласно стратегии черного ящика, используя соответствующие критерии тестирования.

2

Краткое описание методов решения постав- ленных задач

Основная задача программы это правильная *валидация* полей данных, вводимых пользователем. Для решения этой задачи были использованы регулярные выражения на основе стандартной библиотеки из C++.

3

Псевдокод

файл main.cpp:

ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"

ВЫЗОВ функции main()

функция main():

УСТАНОВИТЬ ЛОКАЛЬ: en_RU.UTF-8

ВЫВЕСТИ: Назначение программы

ВЫВЕСТИ: Возможные опции

ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ВВЕЛ EOF:

ОБРАБОТАТЬ ПОТОК ВВОДА ДЛЯ ВЗАИМОДЕЙСТВИЯ С ПРОГРАММОЙ

ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ РАБОТУ С ПРОГРАММОЙ ТО:

ВЫЙТИ ИЗ ЦИКЛА

ОЖИДАТЬ ПОЛЬЗОВАТЕЛЬСКИЙ ВВОД

ВЫВЕСТИ: Сообщение о том, что программа завершена.

ЗАВЕРШИТЬ ПРОГРАММУ

ВЕРНУТЬ 0

файл file_handler.cpp:

ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"

ПРОИНИЦИАЛИЗИРОВАТЬ глобальные переменные: СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1,

СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2, НИЖНЯЯ_ГРАНИЦА

ОБЪЯВИТЬ БУФФЕР[256] // ИЗ 256 ЭЛЕМЕНТОВ

функция clear_stdin():

ОЧИЩАЕТ ПОТОК ВВОДА

функция validate_filename(имя_файла):

ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_файла{"[a-zA-Z0-9_-]+\.\txt"}

ВЕРНУТЬ имя_файла СООТВЕТСТВУЕТ регулярное_выражение_файла

функция validate_license(номерной_знак):

```

ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_номерного_знака{"[АВЕКМНОРСТУХ][0-9]{3,3}
[АВЕКМНОРСТУХ]{2,2}[0-9]{2,3}RUS"}
ВЕРНУТЬ номерной_знак СООТВЕТСТВУЕТ регулярное_выражение_номерного_знака

```

```

функция validate_brand(бренд):
    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_бренда{"[А-ЯЁа-яёА-Za-z0-9-_.! ]+"}
    ВЕРНУТЬ бренд СООТВЕТСТВУЕТ регулярное_выражение_бренда

```

```

функция validate_model(модель):
    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_модели{"[А-ЯЁа-яёА-Za-z0-9-_.!() ]+"}
    ВЕРНУТЬ модель СООТВЕТСТВУЕТ регулярное_выражение_модели

```

```

функция validate_surname(фамилия):
    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_фамилии{"[А-ЯЁа-яёА-Za-z]+"}
    ВЕРНУТЬ фамилия СООТВЕТСТВУЕТ регулярное_выражение_фамилии

```

```

функция validate_address(адрес):
    ПРОИНИЦИАЛИЗИРОВАТЬ регулярное_выражение_адреса{[А-Za-zA-ЯЁа-яё0-9!_-. ]}
    ВЕРНУТЬ адрес СООТВЕТСТВУЕТ регулярное_выражение_адреса

```

```

функция validate_release_year(год_выпуска):
    ПРОИНИЦИАЛИЗИРОВАТЬ результат = ИСТИНА
    ЕСЛИ год_выпуска ПУСТОЙ ИЛИ СОДЕРЖИТ НЕЦИФРЫ ТО:
        ВЫВЕСТИ: Сообщение о неправильном вводе.
        ВЕРНУТЬ ЛОЖЬ

    ПРОИНИЦИАЛИЗИРОВАТЬ текущий_год = ТЕКУЩИЙ ГОД СИСТЕМЫ
    ЕСЛИ год_выпуска < НИЖНЯЯ_ГРАНИЦА ИЛИ год_выпуска > ТЕКУЩИЙ ГОД СИСТЕМЫ ТО:
        ВЫВЕСТИ: "Год вне диапазона"
        результат = ЛОЖЬ
    ВЕРНУТЬ результат

```

```

функция file_info(стандартное_имя_файла):
    ПРИНЯТЬ пользовательский ввод
    ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ имя_файла
    ОБРАБОТАТЬ ПОТОК ВВОДА И ОПРЕДЕЛИТЬ режим_открытия_файла
    ВЕРНУТЬ имя_файла И режим_открытия_файла

```

```

функция write_car_data():
    ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла = file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_1)
    ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла

```

```

ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:
    ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
    ВЕРНУТЬ
    ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"

```

```

ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:
    ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:
        ВЫВЕСТИ: Сообщение об ошибке в введенном поле
        ПОВТОРИТЬ ВВОД ПОЛЯ
    ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:
        ЗАПИСАТЬ ДАННЫЕ В файл
        ВЫВЕСТИ: Сообщение о введенной записи

```

ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:
ЗАВЕРШИТЬ ЦИКЛ

ЗАКРЫТЬ файл
ОЧИСТИТЬ ПОТОК ВВОДА
ВЕРНУТЬ

функция write_license_data():

ПРОИНИЦИАЛИЗИРОВАТЬ имя_файла, режим_открытия_файла = file_info(СТАНДАРТНОЕ_ИМЯ_ФАЙЛА_2)
ПРОИНИЦИАЛИЗИРОВАТЬ файл(имя_файла) В РЕЖИМЕ ОТКРЫТИЯ режим_открытия_файла

ЕСЛИ файл НЕ ОТКРЫВАЕТСЯ ТО:

ВЫВЕСТИ: "Не удалось открыть файл %имя_файла\n"
ВЕРНУТЬ

ВЫВЕСТИ: "Файл %имя_файла успешно открыт для записи\n"

ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД ДАННЫХ:

ЕСЛИ ПОЛЕ ВВЕДЕНО НЕПРАВИЛЬНО ТО:

ВЫВЕСТИ: Сообщение об ошибке в введенном поле
ПОВТОРИТЬ ВВОД ПОЛЯ

ЕСЛИ ВСЕ ПОЛЯ ВВЕДЕНЫ ПРАВИЛЬНО:

ЗАПИСАТЬ ДАННЫЕ В файл

ВЫВЕСТИ: Сообщение о введенной записи

ЕСЛИ ПОЛЬЗОВАТЕЛЬ ЗАВЕРШИЛ ВВОД ДАННЫХ ТО:
ЗАВЕРШИТЬ ЦИКЛ

ЗАКРЫТЬ файл
ОЧИСТИТЬ ПОТОК ВВОДА
ВЕРНУТЬ

4

Сведения о программной реализации

4.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования

Языки программирования: C++17 (GNU C Compiler 15.2.1)

Утилита сборки программы: GNU Make 4.4.1

Системы программирования: Visual Studio Code 1.106.0, Neovim v0.11.5

4.2 Описание входных и выходных данных

Входные данные:

Пользователь выбирает опции функционирования программы и записывает данные в выбранный файл посредством ввода текста. Ввод представляет собой номер выбранной опции – число, имя файла – текст, с которым будет связано дальнейшее выполнение программы, а также последовательный ввод полей для записи в файл – текст

Структура входных данных:

данные вводятся по одному на строку, таким образом пользователь может ввести только одну опцию за раз, а заполнение полей записей файла происходит по одному на строку (одно поле – одна строка)

Выходные данные и их структура:

Пользователь имеет два вида выходных данных:

1. Консольные сообщения: представляют собой текст, необходимый пользователю для пользования программой. Сообщения включают в себя: назначение программы, опции пользования программой, информацию об успешном/неуспешном открытии файла, предупреждения, если пользователь неправильно ввел поле записи, а также сообщение о завершении программы.
2. Файлы: представляют из себя текстовые файлы, содержащие записи, введенные пользователем в порядке записи по одному на строку, имя файла либо вводится пользователем, либо присваивается стандартное программой: `car_data.txt` или `license_data.txt`, в зависимости от назначения программы.

Файл автомобильных сведений:

состоит из записей, каждая из которых содержит три поля – марка, модель и номерной знак.

Файл регистрационных сведений:

состоит из записей, каждая из которых содержит четыре поля – номерной знак, фамилия, адрес владельца и год выпуска.

4.3 Краткое описание спроектированных программных единиц

файл `main.cpp`:

Подключает содержимое «`file_handler.hpp`»

функция `int main()`:

Назначение: является точкой входа исполнения программы, используется для взаимодействия с пользователем.

Параметры: Отсутствуют.

Возвращает: статус завершения программы, тип целое число(`int`).

файл `file_handler.hpp`:

Подключает следующие библиотеки:

- `cstdio` – библиотека стандартного ввода, вывода C
- `string` – библиотека для работы со строками
- `regex` – библиотека для работы с регулярными выражениями
- `cstdlib` – стандартная библиотека C

- filesystem – библиотека для работы с файловой системой, нужна для проверки существования файла

Содержит следующие структуры:

- CarData: – это структура для хранения автомобильных сведений и содержит следующие поля:
 - license – поле содержит номерной знак автомобиля
 - brand – поле содержит бренд автомобиля
 - model – поле содержит модель автомобиля
- LicenseData: – это структура для хранения регистрационных сведений
 - license – поле содержит номерной знак автомобиля
 - surname – поле содержит фамилию владельца автомобиля
 - address – поле содержит адрес владельца автомобиля
 - release_year – поле содержит год выпуска автомобиля

Также содержит все прототипы функций: (подробнее о функциях ниже)

- `void clear_stdin();`
- `bool validate_filename(std::string filename);`
- `bool validate_license(std::string license);`
- `bool validate_brand(std::string brand);`
- `bool validate_model(std::string model);`
- `bool validate_surname(std::string surname);`
- `bool validate_address(std::string address);`
- `bool validate_release_year(std::string release_year);`
- `std::pair<std::string, std::string> file_info(std::string default_filename);`
- `void write_car_data();`
- `void write_license_data();`

файл file_handler.cpp:

Подключает содержимое file_handler.hpp и содержит реализации функций

Под правильностью введенных полей, имен файлов, понимается:

1. в случае полей: это отсутствие *неподходящих символов* и *правильный* порядок элементов.
2. в случае имени файла: это *подходящее* имя файла, которое операционная система способна создать.

Функция	Назначение	Аргументы	Возвращает
clear_stdin	Очистка потока ввода	Отсутствуют	Ничего. Тип: void

Функция	Назначение	Аргументы	Возвращает
validate_filename	Валидация имени файла	filename – имя файла. Тип: std::string	true если имя верно, false если неверно. Тип: bool
validate_license	Валидация поля номерной знак	license – содержимое поля номерной знак. Тип std::string	true если номерной знак введен верно, иначе false. Тип: bool
validate_brand	Валидация поля бренд	brand – содержимое поля бренд. Тип: std::string	true если бренд введен верно, иначе false. Тип: bool
validate_model	Валидация поля модель	model – содержимое поля модель. Тип: std::string	true если модель введена верно, иначе false. Тип: bool
validate_surname	Валидация поля бренд	surname – содержимое поля фамилия. Тип: std::string	true, если фамилия введена верно, иначе false. Тип: bool
validate_address	Валидация поля адрес	address – содержимое поля адрес. Тип: std::string	true, если адрес введен верно, иначе false. Тип: bool
validate_release_year	Валидация поля год выпуска	release_year – содержимое поля год выпуска. Тип: std::string	true если год выпуска введен верно и подходит по временному интервалу, иначе false. Тип: bool
file_info	Спрашивает у пользователя имя файла и режим работы с ним, для последующей работы с файлом	default_file_name – стандартное имя файла, в зависимости от выбранной опции, принимает соответствующее стандартное имя файла. Тип: std::string	пару значений, первое – это имя файла, второе – это режим открытия файла. Тип: std::pair<std::string, std::string>
write_car_data	Запись введенных пользователем сведений в файл автомобильных сведений, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void

Функция	Назначение	Аргументы	Возвращает
write_license_data	Запись введенных пользователем сведений в файл регистрации, учитывая проверки на корректность ввода	Отсутствуют	Ничего Тип: void

файл make_fileh:

Содержит алгоритм сборки программного решения, а также алгоритм очистки созданных в результате компиляции и работы программы файлов.

4.4 Список диагностических сообщений с указанием возможных причин их появления

- Сообщение: «Введен %с программа продолжает работу\n»
Причина: Введенный символ не является N,n,Q,q или «enter»('\n').
- Сообщение: «Введен %с. Можно ввести только опцию 1 или опцию 2. Попробуйте еще раз.»
Причина: Введенный символ не является 1 или 2.
- Сообщение: «Введена пустая строка».
Причина: Пользователь ввел enter('\n') или что-то с нулевой длиной в поле год выпуска.
- Сообщение: «Строка не полностью состоит из цифр».
Причина: Поле год, введенное пользователем, содержит что-то кроме цифр.
- Сообщение: «Год вне диапазона(%d-%d)».
Причина: Поле год, введенное пользователем, выходит за описанные временные рамки.
- Сообщение: «Введенное имя файла некорректно. Файл должен иметь вид file.txt, где file состоит из латинских символов, дефиса(-) или нижнего подчеркивания(_), а после следует его расширение .txt».
Причина: Пользователь ввел неподходящее имя файла.
- Сообщение: «Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?».
Причина: Пользователь ввел имя файла, который уже существует.
- Сообщение: «Не удалось открыть файл %s\n». Причина: Файл может не открываться, если неправильно введено имя файла, или с ним уже происходит работа.

- Сообщение: «Поле введено некорректно, попробуйте еще раз\n» Причина: Поле не соответствует заданному примеру/форме или заданному множеству элементов.

5

Инструкция использования программы

Инструкция полностью соответствует указаниям на экране.

Но все же рассмотрим как пользоваться программой:

Для компиляции программы пользователь должен ввести:

```
make -f make_fileh
```

Для запуска программы пользователь должен запустить файл ./file_h

Затем пользователь увидит следующее:

```
Назначение программы: Создание файлов на основе сведений, вводимых пользователем.  
Что требуется записать?  
1) Файл автомобильных сведений  
2) Файл регистрационных сведений  
q) Для завершения программы нажмите q.
```

Теперь у пользователя есть выбор, как взаимодействовать с программой.

Рассмотрим основные варианты:

Допустим пользователь ввел 1. Теперь он увидит следующее:

```
Введите название файла(enter=car_data.txt):
```

Программа ожидает от пользователя название файла, в который будут записаны сведения. Если пользователь введет enter, то он будет работать с файлом car_data.txt. Если пользователь введет любое другое имя файла, которое удовлетворяет требованиям, например file1.txt, он увидит следующие сообщения:

Файл file1.txt успешно открыт для записи.
Начинаем заполнять автомобильные сведения.
Для завершения ввода нажмите 'Q' или 'q'.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Но если файл file1.txt уже существует будет выведено следующее сообщение:

Файл file1.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

В таком случае программа в зависимость от ввода пользователя будет либо дописывать сведения в файл, либо заполнит его с нуля.

Теперь программа ожидает от пользователя заполнение поля бренд.

Если пользователь заполнит его без ошибок, то он перейдет к записи следующего поля. Если же пользователь совершит ошибку при вводе, то он увидит следующее сообщение:

Поле введено некорректно, попробуйте еще раз.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Это сообщение будет повторяться до тех пор, пока пользователь не введет поле правильно, или не решит завершить ввод, нажав q или Q. Как только пользователь правильно введет поле, он перейдет к заполнению следующего и так далее, пока не завершит ввод сведений, нажатием клавиши q или Q.

Стоит отметить, что данные записываются в файл при условии, что все каждое из полей было введено верно и при этом, все поля были заполнены.

Как только пользователь заполнит первую запись он увидит сообщение:

Запись 1 завершена:
бренд: Toyota
модель: Supra
номер: M976MM777RUS
Продолжаем ввод сведений.
Введите поле бренд (символы: А-ЯЁа-яА-Яа-я0-9-_.!):

Если пользователь введет q или Q он увидит следующее сообщение:

Желаете продолжить запись? (enter=Да/N=нет)

Теперь программа ожидает от пользователя выбор либо продолжить запись, завершить работу с программой. Если пользователь введет enter, он снова увидит сообщение о выборе опций взаимодействия с программой.

А если пользователь решит завершить работу с программой он увидит данное сообщение:

Нажмите любую клавишу для завершения программы...

Затем после нажатия любой клавиши, будет выведено следующее сообщение:

Программа завершена.

6

Листинг программной разработки

файл main.cpp:

```
#include "file_handler.hpp"

int main() {
    setlocale(LC_ALL, "en_RU.UTF-8");
    printf("Назначение программы: Создание файлов на основе сведений, вводимых\nпользователем\n");
    printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл\nрегистрационных сведений\n");
    printf("q) Для завершения программы нажмите q.\n");
    char input;
    while ((input = getchar()) != EOF) {
        clear_stdin();
        if (input == '1' || input == '2') {
            if (input == '1') write_car_data();
            else
                if (input == '2') write_license_data();
        } else if (input == 'q' || input == 'Q') {
            break;
        }
        else printf("Данной опции не существует.\n");

        printf("Желаете продолжить запись? (enter=Да/N=Нет)\n");
        input = getchar();
        clear_stdin();
        if (input == 'N' || input == 'n') {
            break;
        } else
            if (input == '\n') {
                else printf("Введен enter, программа продолжает работу\n");
            } else printf("Введен %c, программа продолжает работу\n", input);
        printf("Что требуется записать?\n1) Файл автомобильных сведений\n2) Файл\nрегистрационных сведений\n");
        printf("q) Для завершения программы нажмите q.\n");
    }
}
```

```

    }
    printf("Нажмите любую клавишу для завершения программы...");
    scanf("%c", &input);
    printf("Программа завершена.\n");
    return 0;
}

```

файл file_handler.hpp:

```

#ifndef FILE_HANDLER_HPP
#define FILE_HANDLER_HPP

#include <cstdio>
#include <string>
#include <regex>
#include <cstdlib>
#include <filesystem>

struct CarData {
    std::string license;
    std::string brand;
    std::string model;
};

struct LicenseData {
    std::string license;
    std::string surname;
    std::string address;
    std::string release_year;
};

void clear_stdin();
bool validate_filename(std::string filename);
bool validate_license(std::string license);
bool validate_brand(std::string brand);
bool validate_model(std::string model);
bool validate_surname(std::string surname);
bool validate_address(std::string address);
bool validate_release_year(std::string release_year);

std::pair<std::string, std::string> file_info(std::string default_filename);
void write_car_data();
void write_license_data();

#endif

```

файл file_handler.cpp

```

#include "file_handler.hpp"

std::string DEFAULT_FILENAME_1 = "car_data.txt";
std::string DEFAULT_FILENAME_2 = "license_data.txt";
const int LOWER_BOUND_YEAR = 1960;

```

```

char BUFFER[256];

void clear_stdin() {
    int temp;
    while ((temp = getchar()) != EOF && temp != '\n');
}

bool validate_filename(std::string filename) {
    std::regex regex_filename{"[a-zA-Z0-9_-]+\\.txt"};
    return std::regex_match(filename, regex_filename);
}

bool validate_license(std::string license) {
    // M976MM777RUS
    std::regex pattern{"[АВЕКМНОРСТУХ][0-9]{3,3}[АВЕКМНОРСТУХ]{2,2}[0-9]{2,3}RUS"};
    return regex_match(license, pattern);
}

bool validate_brand(std::string brand) {
    std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.! ]+"};
    return regex_match(brand, pattern);
}

bool validate_model(std::string model) {
    std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9-_.!() ]+"};
    return regex_match(model, pattern);
}

bool validate_surname(std::string surname) {
    std::regex pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z ]+"};
    return regex_match(surname, pattern);
}

bool validate_address(std::string address) {
    std::regex address_pattern{"[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-Za-z0-9!_. ]+"};
    return regex_match(address, address_pattern);
}

bool validate_release_year(std::string release_year) {
    bool result=true;
    if (release_year.size()==0) {
        printf("Введена пустая строка.\n");
        return false;
    }
    for (int i=0; i<release_year.size(); ++i) {
        if (!isdigit(release_year[i])) {
            printf("Строка не полностью состоит из цифр.\n");
            return false;
        }
    }
    int year=std::stoi(release_year);
    std::time_t t = std::time(nullptr);

```

```

std::tm *const pTInfo = std::localtime(&t);
int current_year = 1900 + pTInfo->tm_year;
if (!(year >= LOWER_BOUND_YEAR && year <= current_year) || year <= 0) {
    printf("Год вне диапазона(%d-%d)\n", LOWER_BOUND_YEAR, current_year);
    result = false;
}
return result;
}

std::pair<std::string, std::string> file_info(std::string default_file_name) {
    std::pair<std::string, std::string> file_info;
    std::string file_name;
    printf("Введите название файла(enter=%s): ", default_file_name.c_str());
    while (scanf("%255[^\n]s", BUFFER)) {
        clear_stdin();
        file_name = BUFFER;
        if (validate_filename(file_name)) {
            break;
        }
        else {
            printf("Введенное имя файла некорректно. Файл должен иметь вид file.txt, \n");
            printf("где file состоит из латинских символов, дефиса(-) или нижнего
подчеркивания(_),\n");
            printf("а после следует его расширение .txt\n");
            printf("Введите название файла(enter=%s): ", default_file_name.c_str());
        }
    }
    if (file_name.size() == 0) {
        file_name = default_file_name;
    }
    file_info.first = file_name;
    if (std::filesystem::exists(file_name)) {
        printf("Файл %s уже существует.\nЖелаете продолжить запись вместо заполнения с
начала(enter=Да/Н=Нет)?\n", file_name.c_str());
        while ((BUFFER[0]=getchar()) != EOF) {
            clear_stdin();
            if (BUFFER[0] == '\n') {
                file_info.second = "a";
                break;
            }
            else {
                if (BUFFER[0] == 'n' || BUFFER[0] == 'N') {
                    file_info.second = "w";
                    break;
                }
                else printf("Введено '%c' можно ввести только enter или N\n", BUFFER[0]);
            }
        }
    }
    else file_info.second = "w";
    return file_info;
}

void write_car_data() {
    std::pair<std::string, std::string> file_information = file_info(DEFAULT_FILENAME_1);
    std::string file_name = file_information.first;
    FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());
}

```

```

if (!file) {
    printf("Не удалось открыть файл %s\n", file_name.c_str());
    return;
}
printf("Файл %s успешно открыт для записи\n", file_name.c_str());

int field_index=0;
CarData temp = {};
printf("Начинаем заполнять автомобильные сведения.\nДля завершения ввода нажмите 'Q'
либо 'q'.\n");
std::string field_names[3] = {"бренд (символы: A-ЯЁа-яёA-Za-z0-9-_.! )", "модель
(символы: A-ЯЁа-яёA-Za-z0-9-_.!() )", "номерной знак (пример: M976MM777RUS)"};
printf("Введите поле %s:\n", field_names[field_index].c_str());
bool filled = false;
int note_number = 0;
while (scanf("%255[^\n]s", BUFFER) != EOF) {
    if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
        printf("Введен %s. Ввод сведений закончен\n", BUFFER);
        clear_stdin();
        break;
    }
    clear_stdin();
    switch (field_index) {
        case 0: {
            temp.brand.assign(BUFFER);
            if (validate_brand(temp.brand))
                field_index=(field_index+1)%3;
            else printf("Поле введено некорректно, попробуйте еще раз\n");
            break;
        }
        case 1: {
            temp.model.assign(BUFFER);
            if (validate_model(temp.model))
                field_index=(field_index+1)%3;
            else printf("Поле введено некорректно, попробуйте еще раз\n");
            break;
        }
        case 2: {
            temp.license.assign(BUFFER);
            if (validate_license(temp.license)) {
                field_index = (field_index+1)%3;
                filled = true;
            } else printf("Поле введено некорректно, попробуйте еще раз\n");
            break;
        }
    }
}
if (filled) {
    fprintf(file, "%s %s %s\n", temp.brand.c_str(), temp.model.c_str(),
temp.license.c_str());
    printf(
        "Запись #%d завершена:\nбренд: %s\nмодель: %s\nномерной знак: %s\n",
        ++note_number,

```

```

        temp.brand.c_str(),
        temp.model.c_str(),
        temp.license.c_str()
    );
    printf("Продолжаем запись сведений\n");
    temp = {};
    filled = false;
}
printf("Введите поле %s:\n", field_names[field_index].c_str());
}
fclose(file);
// clear stdin to ignore EOF
clearerr(stdin);
return;
}

void write_license_data() {
    std::pair<std::string, std::string> file_information = file_info(DEFAULT_FILENAME_2);
    std::string file_name = file_information.first;
    FILE* file = std::fopen(file_name.c_str(), file_information.second.c_str());

    if (!file) {
        printf("Не удалось открыть файл %s\n", file_name.c_str());
        return;
    }
    printf("Файл %s успешно открыт для записи\n", file_name.c_str());

    int field_index=0;
    LicenseData temp = {};

    printf("Начинаем заполнять регистрационные сведения.\nДля завершения ввода нажмите 'Q'
либо 'q'.\n");
    std::string field_names[4] = {"номерной знак (пример: M976MM777RUS)", "фамилия
владельца (символы: А-ЯЁа-яёА-Za-z)", "адрес владельца (символы: А-Za-zA-ЯЁа-яё0-9!_-. )",
"год выпуска (символы: 0-9)"};
    printf("Введите поле %s:\n", field_names[field_index].c_str());
    bool filled = false;
    int note_number = 0;
    while (scanf("%255[^\n]s", BUFFER) != EOF) {
        if ((BUFFER[0]=='Q' || BUFFER[0] == 'q') && BUFFER[1] == '\0') {
            printf("Введен %s. Ввод сведений закончен\n", BUFFER);
            clear_stdin();
            break;
        }
    }
    clear_stdin();
    switch (field_index) {
        case 0: {
            temp.license.assign(BUFFER);
            if (validate_license(temp.license)) {
                field_index = (field_index+1)%4;
            } else printf("Поле введено некорректно, попробуйте еще раз\n");
            break;
        }
    }
}

```

```

    case 1: {
        temp.surname.assign(BUFFER);
        if (validate_surname(temp.surname)) {
            field_index = (field_index+1)%4;
        } else printf("Поле введено некорректно, попробуйте еще раз\n");
        break;
    }
    case 2: {
        temp.address.assign(BUFFER);
        if (validate_address(temp.address)) {
            field_index = (field_index+1)%4;
        } else printf("Поле введено некорректно, попробуйте еще раз\n");
        break;
    }
    case 3: {
        temp.release_year.assign(BUFFER);
        if (validate_release_year(temp.release_year)) {
            field_index = (field_index+1)%4;
            filled = true;
        } else printf("Поле введено некорректно, попробуйте еще раз\n");
        break;
    }
}

if (filled) {
    fprintf(file, "%s %s %s %s\n", temp.license.c_str(), temp.surname.c_str(),
temp.address.c_str(), temp.release_year.c_str());
    printf(
        "Запись #%d завершена:\nномерной знак: %s\nфамилия владельца: %s\nадрес:
%s\nгод выпуска: %s\n",
        ++note_number,
        temp.license.c_str(),
        temp.surname.c_str(),
        temp.address.c_str(),
        temp.release_year.c_str()
    );

    printf("Продолжаем запись сведений\n");
    temp = {};
    filled = false;
}
printf("Введите поле %s:\n", field_names[field_index].c_str());
}
fclose(file);
// clear stdin to ignore EOF
clearerr(stdin);
return;
}

```

файл make_fileh:

```

all: main.o file_handler.o
    g++ -g main.o file_handler.o -o file_h

```

```
main.o: main.cpp
  g++ -g -c main.cpp
file_handler.o: file_handler.cpp
  g++ -c file_handler.cpp -std=c++17
clean:
  rm -f *.o *.out *.exe *.txt
```

7

Результаты тестирования

Для демонстрации работы программы заполним файл car_data.txt и выведем его содержимое на экран:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
1
Введите название файла(enter=car_data.txt):
Файл car_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл car_data.txt успешно открыт для записи
Начинаем заполнять автомобильные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! )):
Dodge
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() )):
Charger 1969
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Запись #1 завершена:
бренд: Dodge
модель: Charger 1969
номерной знак: M977MM777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! )):
Toyota
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() )):
Supra
Введите поле номерной знак (пример: M976MM777RUS):
M976MM766RUS
Запись #2 завершена:
бренд: Toyota
модель: Supra
номерной знак: M976MM766RUS
```

Рис. 1. Пример заполнения первых 2-х записей.

Введем еще одну запись и завершим ввод сведений в файл.

```
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
Мерседес
Введите поле модель (символы: А-ЯЁа-яёА-Za-z0-9-_.!() ) :
Модель S
Введите поле номерной знак (пример: M976MM777RUS):
M976MM777RUS
Запись #3 завершена:
бренд: Мерседес
модель: Модель S
номерной знак: M976MM777RUS
Продолжаем запись сведений
Введите поле бренд (символы: А-ЯЁа-яёА-Za-z0-9-_.! ) :
q
Введен q. Ввод сведений закончен
```

Рис. 2. Пример заполнения еще одной записи.

Опять видим основные опции программы. Завершим работу с программой.

```
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
q
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 3. Снова основное меню.

Выведем на экран содержимое файла car_data.txt.

```
→ cat car_data.txt
Dodge Charger 1969 M977MM777RUS
Toyota Supra M976MM766RUS
Мерседес Модель S M976MM777RUS
```

Рис. 4. Содержимое car_data.txt.

Можно заметить, что информация записалась в файл без каких либо проблем.

Теперь запишем информацию в файл license_data.txt:

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt): license_data.txt
Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M977MM777RUS
Введите поле фамилия владельца (символы: A-ЯЁа-яёA-Za-z):
Pushkin
Введите поле адрес владельца (символы: A-Za-zA-ЯЁа-яё0-9!_-. ):
Ул. Новые Черемушки
Введите поле год выпуска (символы: 0-9):
2008
Запись #1 завершена:
номерной знак: M977MM777RUS
фамилия владельца: Pushkin
адрес: Ул. Новые Черемушки
год выпуска: 2008
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 5. Пример заполнения license_data.txt.

Теперь выведем содержимое файла на экран.

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
```

Рис. 6. Содержимое license_data.txt.

Теперь допишем информацию в license_data.txt.

```
→ ./file_h
Назначение программы: Создание файлов на основе сведений, вводимых пользователем
Что требуется записать?
1) Файл автомобильных сведений
2) Файл регистрационных сведений
q) Для завершения программы нажмите q.
2
Введите название файла(enter=license_data.txt):
Файл license_data.txt уже существует.
Желаете продолжить запись вместо заполнения с начала(enter=Да/N=Нет)?

Файл license_data.txt успешно открыт для записи
Начинаем заполнять регистрационные сведения.
Для завершения ввода нажмите 'Q' либо 'q'.
Введите поле номерной знак (пример: M976MM777RUS):
M976CC777RUS
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Za-z):
&
Поле введено некорректно, попробуйте еще раз
Введите поле фамилия владельца (символы: А-ЯЁа-яёА-Za-z):
Nikitin
Введите поле адрес владельца (символы: А-Za-zА-ЯЁа-яё0-9!_-. ):
Улица Профсоюзная д.26
Введите поле год выпуска (символы: 0-9):
1977
Запись #1 завершена:
номерной знак: M976CC777RUS
фамилия владельца: Nikitin
адрес: Улица Профсоюзная д.26
год выпуска: 1977
Продолжаем запись сведений
Введите поле номерной знак (пример: M976MM777RUS):
q
Введен q. Ввод сведений закончен
Желаете продолжить запись? (enter=Да/N=Нет)
n
Нажмите любую клавишу для завершения программы...
Программа завершена.
```

Рис. 7. Пример дозаписи сведений в license_data.txt.

Стоит отметить, что при заполнении поля фамилия была совершена ошибка. Программа заметила, что поле введено некорректно, и потребовала перезаписать поле.

Теперь посмотрим содержимое license_data.txt

```
→ cat license_data.txt
M977MM777RUS Pushkin Ул. Новые Черемушки 2008
M976CC777RUS Nikitin Улица Профсоюзная д.26 1977
```

Рис. 8. Содержимое license_data.txt после дозаписи.

В результате небольшого тестирования можно отметить, что программа функционирует корректно.

8

Дальнейшее развитие программы.

Несмотря на то, что программа справляется со своими задачами, у нее хватает недостатков.

Один из таких недостатков это интерфейс. Взаимодействие с программой через консоль не самый удобный вариант.

Также стоит отметить, что программа имеет очень ограниченный функционал, например поддерживает работу только с текстовыми файлами, но при этом не позволяет пользователю редактировать файл, а дает либо дописать информацию, либо перезаписать её вовсе.

В дальнейших версиях программы эти недочеты будут исправлены.